Formative Assessment 6

PATAYON, SPIKE LEE-ROY V

2026-04-14

Instructions

1. Download the dataset: **Customer Churn Dataset.**.
2. Perform the analysis using **Python (Jupyter Notebook), R Markdown, or any Markdown-supported environment**.
3. You are required to implement a **Neural Network model** based on the concepts discussed in class (hidden layers, activation functions, SGD, etc.).
4. Clearly show:
    - Code
    - Outputs (tables/plots)
    - Brief explanations (1–3 sentences per task)
5. Submit your work, **group submission allowed, max 2 members per group under FA 6 Group**.
6. File format: **PDF or HTML export** of your notebook/markdown.

```r
library(ggplot2)
library(keras3)
```

```
## Warning: package 'keras3' was built under R version 4.5.3
```

Assessment Tasks

a. Data Preparation (5 points)

- Load the dataset and display the first 10 observations.

```r
df = read.csv("C:\\Users\\spike\\Downloads\\WA_Fn-UseC_-Telco-Customer-Churn.csv")

head(df,10)
```

```
##    customerID gender SeniorCitizen Partner Dependents tenure PhoneService
## 1  7590-VHVEG Female             0     Yes         No      1           No
## 2  5575-GNVDE   Male             0      No         No     34          Yes
## 3  3668-QPYBK   Male             0      No         No      2          Yes
## 4  7795-CFOCW   Male             0      No         No     45           No
## 5  9237-HQITU Female             0      No         No      2          Yes
## 6  9305-CDSKC Female             0      No         No      8          Yes
## 7  1452-KIOVK   Male             0      No        Yes     22          Yes
## 8  6713-OKOMC Female             0      No         No     10           No
## 9  7892-POOKP Female             0     Yes         No     28          Yes
## 10 6388-TABGU   Male             0      No        Yes     62          Yes
##       MultipleLines InternetService OnlineSecurity OnlineBackup
## 1  No phone service             DSL             No          Yes
## 2                No             DSL            Yes           No
## 3                No             DSL            Yes          Yes
## 4  No phone service             DSL            Yes           No
## 5                No     Fiber optic             No           No
## 6               Yes     Fiber optic             No           No
## 7               Yes     Fiber optic             No          Yes
## 8  No phone service             DSL            Yes           No
## 9               Yes     Fiber optic             No           No
## 10               No             DSL            Yes          Yes
##    DeviceProtection TechSupport StreamingTV StreamingMovies       Contract
## 1                No          No          No              No Month-to-month
## 2               Yes          No          No              No       One year
## 3                No          No          No              No Month-to-month
## 4               Yes         Yes          No              No       One year
## 5                No          No          No              No Month-to-month
## 6               Yes          No         Yes             Yes Month-to-month
## 7                No          No         Yes              No Month-to-month
## 8                No          No          No              No Month-to-month
## 9               Yes         Yes         Yes             Yes Month-to-month
## 10               No          No          No              No       One year
##    PaperlessBilling             PaymentMethod MonthlyCharges TotalCharges Churn
## 1               Yes          Electronic check          29.85        29.85    No
## 2                No              Mailed check          56.95      1889.50    No
## 3               Yes              Mailed check          53.85       108.15   Yes
## 4                No Bank transfer (automatic)          42.30      1840.75    No
## 5               Yes          Electronic check          70.70       151.65   Yes
## 6               Yes          Electronic check          99.65       820.50   Yes
## 7               Yes   Credit card (automatic)          89.10      1949.40    No
## 8                No              Mailed check          29.75       301.90    No
## 9               Yes          Electronic check         104.80      3046.05   Yes
## 10               No Bank transfer (automatic)          56.15      3487.95    No
```

Quick summary for the whole dataset

```
summary(df)
```

```
##   customerID           gender          SeniorCitizen      Partner
##  Length:7043        Length:7043        Min.   :0.0000   Length:7043
##  Class :character   Class :character   1st Qu.:0.0000   Class :character
##  Mode  :character   Mode  :character   Median :0.0000   Mode  :character
##                                        Mean   :0.1621
##                                        3rd Qu.:0.0000
##                                        Max.   :1.0000
##
##   Dependents            tenure      PhoneService       MultipleLines
##  Length:7043        Min.   : 0.00   Length:7043        Length:7043
##  Class :character   1st Qu.: 9.00   Class :character   Class :character
##  Mode  :character   Median :29.00   Mode  :character   Mode  :character
##                     Mean   :32.37
##                     3rd Qu.:55.00
##                     Max.   :72.00
##
##  InternetService    OnlineSecurity     OnlineBackup       DeviceProtection
##  Length:7043        Length:7043        Length:7043        Length:7043
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##
##  TechSupport        StreamingTV        StreamingMovies      Contract
##  Length:7043        Length:7043        Length:7043        Length:7043
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##
##  PaperlessBilling   PaymentMethod      MonthlyCharges    TotalCharges
##  Length:7043        Length:7043        Min.   : 18.25   Min.   :  18.8
##  Class :character   Class :character   1st Qu.: 35.50   1st Qu.: 401.4
##  Mode  :character   Mode  :character   Median : 70.35   Median :1397.5
##                                        Mean   : 64.76   Mean   :2283.3
##                                        3rd Qu.: 89.85   3rd Qu.:3794.7
##                                        Max.   :118.75   Max.   :8684.8
##                                                         NA's   :11
##     Churn
##  Length:7043
##  Class :character
##  Mode  :character
##
##
##
##
```

```
str(df)
```

```
## 'data.frame':    7043 obs. of  21 variables:
##  $ customerID      : chr  "7590-VHVEG" "5575-GNVDE" "3668-QPYBK" "7795-CFOCW" ...
##  $ gender          : chr  "Female" "Male" "Male" "Male" ...
##  $ SeniorCitizen   : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ Partner         : chr  "Yes" "No" "No" "No" ...
##  $ Dependents      : chr  "No" "No" "No" "No" ...
##  $ tenure          : int  1 34 2 45 2 8 22 10 28 62 ...
##  $ PhoneService    : chr  "No" "Yes" "Yes" "No" ...
##  $ MultipleLines   : chr  "No phone service" "No" "No" "No phone service" ...
##  $ InternetService : chr  "DSL" "DSL" "DSL" "DSL" ...
##  $ OnlineSecurity  : chr  "No" "Yes" "Yes" "Yes" ...
##  $ OnlineBackup    : chr  "Yes" "No" "Yes" "No" ...
##  $ DeviceProtection: chr  "No" "Yes" "No" "Yes" ...
##  $ TechSupport     : chr  "No" "No" "No" "Yes" ...
##  $ StreamingTV     : chr  "No" "No" "No" "No" ...
##  $ StreamingMovies : chr  "No" "No" "No" "No" ...
##  $ Contract        : chr  "Month-to-month" "One year" "Month-to-month" "One year" ...
##  $ PaperlessBilling: chr  "Yes" "No" "Yes" "No" ...
##  $ PaymentMethod   : chr  "Electronic check" "Mailed check" "Mailed check" "Bank transfer (automatic)"
...
##  $ MonthlyCharges  : num  29.9 57 53.9 42.3 70.7 ...
##  $ TotalCharges    : num  29.9 1889.5 108.2 1840.8 151.7 ...
##  $ Churn           : chr  "No" "No" "Yes" "No" ...
```

- Identify the **target variable (Churn)**.

```
table(df$Churn)
```

```
## 
##   No  Yes 
## 5174 1869
```

- Handle missing values.

```
colSums(is.na(df))
```

```
##       customerID           gender    SeniorCitizen          Partner 
##                0                0                0                0 
##       Dependents           tenure     PhoneService    MultipleLines 
##                0                0                0                0 
##  InternetService   OnlineSecurity     OnlineBackup DeviceProtection 
##                0                0                0                0 
##      TechSupport      StreamingTV  StreamingMovies         Contract 
##                0                0                0                0 
## PaperlessBilling    PaymentMethod   MonthlyCharges     TotalCharges 
##                0                0                0               11 
##            Churn 
##                0
```

We have 11 missing values in the column TotalCharges

```
df$TotalCharges = as.numeric(df$TotalCharges)
df = na.omit(df)
```

```
colSums(is.na(df))
```

```
##       customerID           gender    SeniorCitizen          Partner 
##                0                0                0                0 
##       Dependents           tenure     PhoneService    MultipleLines 
##                0                0                0                0 
##  InternetService   OnlineSecurity     OnlineBackup DeviceProtection 
##                0                0                0                0 
##      TechSupport      StreamingTV  StreamingMovies         Contract 
##                0                0                0                0 
## PaperlessBilling    PaymentMethod   MonthlyCharges     TotalCharges 
##                0                0                0                0 
##            Churn 
##                0
```

- Encode categorical variables (e.g., gender, contract type).

  Before we start encoding categorical variables, we must look at the dataset and see what can we fix first

  Convert SeniorCitizen (0/1 → Yes/No)

```
df$SeniorCitizen = ifelse(df$SeniorCitizen == 1, "Yes", "No")
```

```
catvar = c("gender", "SeniorCitizen", "Partner", "Dependents",
              "PhoneService", "MultipleLines", "InternetService",
              "OnlineSecurity", "OnlineBackup", "DeviceProtection",
              "TechSupport", "StreamingTV", "StreamingMovies",
              "Contract", "PaperlessBilling", "PaymentMethod", "Churn")

df[catvar] = lapply(df[catvar], as.factor)
```

- Normalize or standardize numerical variables.

```
numvar = c("tenure", "MonthlyCharges", "TotalCharges")

df[numvar] = scale(df[numvar])
```

```
head(df)
```

```
##   customerID gender SeniorCitizen Partner Dependents      tenure PhoneService
## 1 7590-VHVEG Female            No     Yes         No -1.28015700           No
## 2 5575-GNVDE   Male            No      No         No  0.06429811          Yes
## 3 3668-QPYBK   Male            No      No         No -1.23941594          Yes
## 4 7795-CFOCW   Male            No      No         No  0.51244982           No
## 5 9237-HQITU Female            No      No         No -1.23941594          Yes
## 6 9305-CDSKC Female            No      No         No -0.99496955          Yes
##      MultipleLines InternetService OnlineSecurity OnlineBackup DeviceProtection
## 1 No phone service             DSL             No          Yes               No
## 2               No             DSL            Yes           No              Yes
## 3               No             DSL            Yes          Yes               No
## 4 No phone service             DSL            Yes           No              Yes
## 5               No     Fiber optic             No           No               No
## 6              Yes     Fiber optic             No           No              Yes
##   TechSupport StreamingTV StreamingMovies       Contract PaperlessBilling
## 1          No          No              No Month-to-month              Yes
## 2          No          No              No       One year               No
## 3          No          No              No Month-to-month              Yes
## 4         Yes          No              No       One year               No
## 5          No          No              No Month-to-month              Yes
## 6          No         Yes             Yes Month-to-month              Yes
##             PaymentMethod MonthlyCharges TotalCharges Churn
## 1          Electronic check     -1.1616113   -0.9941234    No
## 2              Mailed check     -0.2608594   -0.1737275    No
## 3              Mailed check     -0.3638974   -0.9595809   Yes
## 4 Bank transfer (automatic)     -0.7477972   -0.1952338    No
## 5          Electronic check      0.1961642   -0.9403906   Yes
## 6          Electronic check      1.1584066   -0.6453233   Yes
```

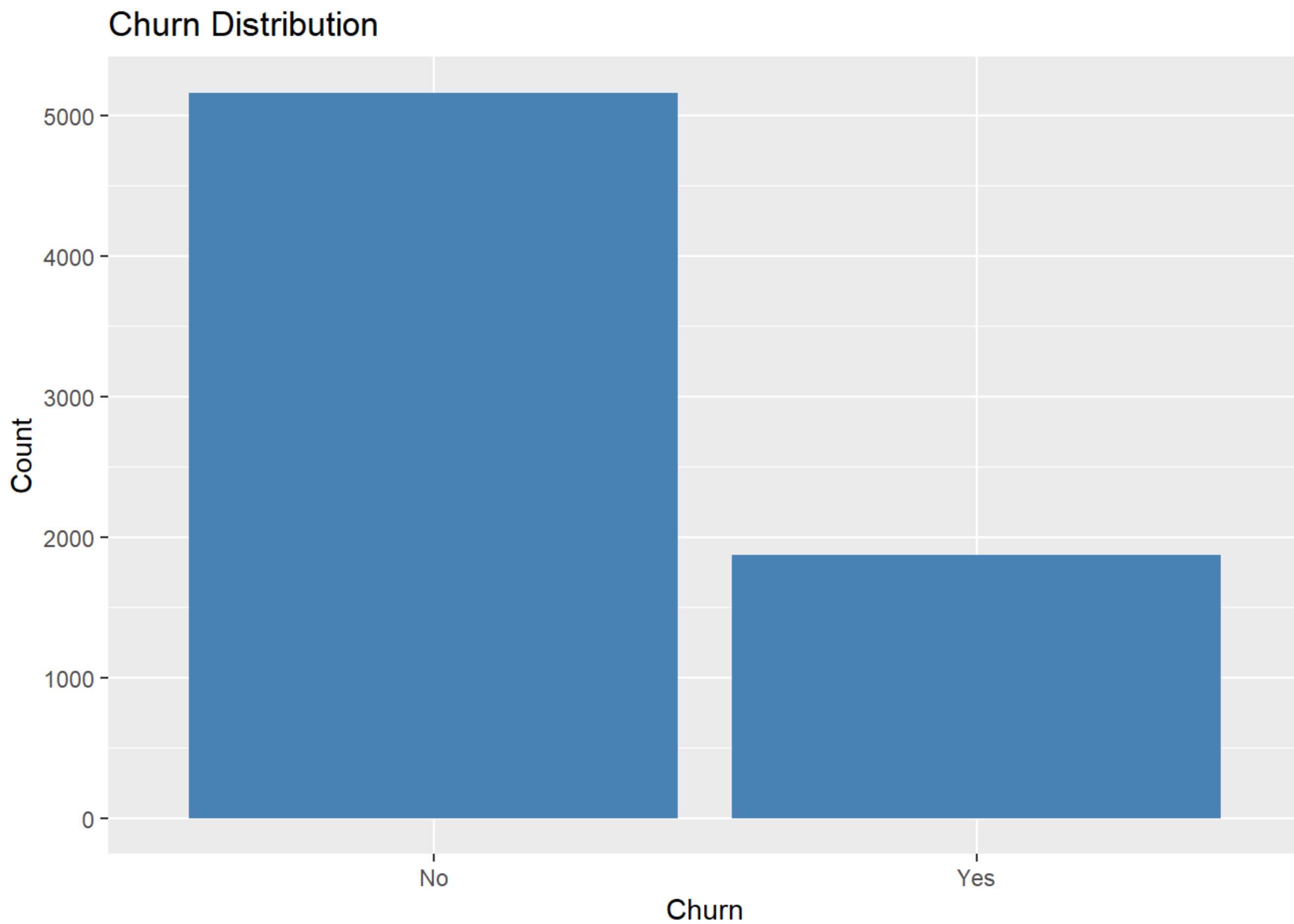
Numerical variables are standardized using scaling so that they have mean 0 and standard deviation 1, improving model performance.

---

b. Exploratory Data Analysis (5 points)

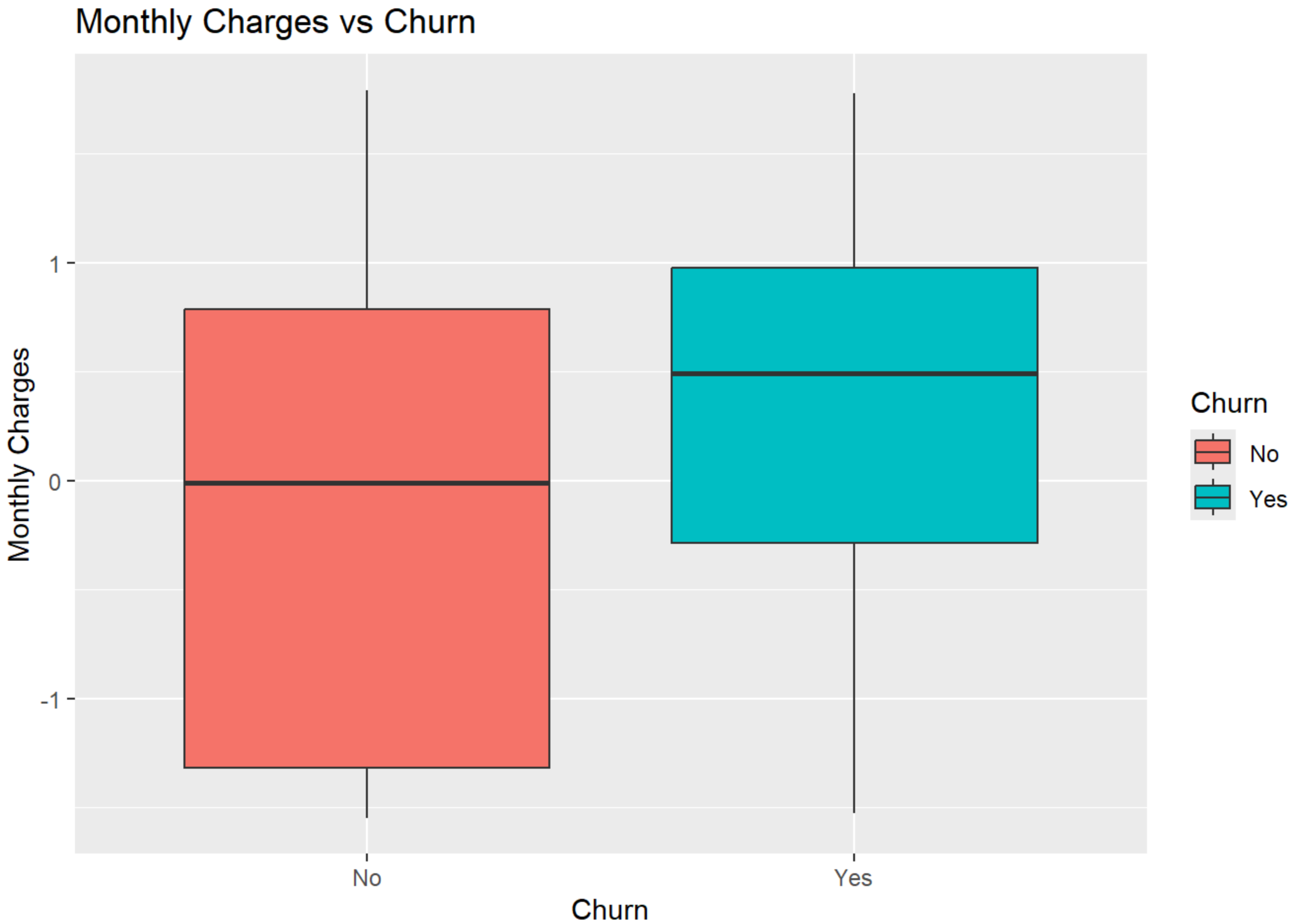
- Create at least **2 visualizations**, such as:
  - Churn distribution

```
ggplot(df, aes (x = Churn)) +
  geom_bar(fill = "steelblue") +
  labs(title = "Churn Distribution",
  x = "Churn",
  y = "Count")
```



- Monthly charges vs churn

```
ggplot(df, aes(x = Churn, y = MonthlyCharges, fill = Churn)) +
  geom_boxplot() +
  labs(title = "Monthly Charges vs Churn",
       x = "Churn",
       y = "Monthly Charges")
```



This boxplot compares monthly charges across churn groups to see if higher costs are linked to churn.

- Identify:
    - Class imbalance (if any)
    - At least 2 variables that may influence churn

Some notable findings

1. Class Imbalance

- There is class imbalance, as non-churn customers are typically more frequent than churn customers.

2. Variables Influencing Churn

- MonthlyCharges: Customers with higher charges tend to churn more.
- Contract: Customers with shorter contracts (e.g., month-to-month) are more likely to churn.

---

c. Neural Network Model Construction (8 points)

- Build a **one-hidden-layer neural network** with:
    - Input layer (features)

```r
# Convert target to numeric (0/1)
df$Churn <- ifelse(df$Churn == "Yes", 1, 0)

# Split features and target
x <- model.matrix(Churn ~ . -1, data = df)
y <- df$Churn

# Define model
model <- keras_model_sequential() %>%
  layer_dense(units = 16, activation = "relu", input_shape = ncol(x)) %>%
  layer_dense(units = 1, activation = "sigmoid")

# Compile model
model %>% compile(
  loss = "binary_crossentropy",
  optimizer = "adam",
  metrics = c("accuracy")
)

model
```

```
## Model: "sequential"
## ┌─────────────────────────────────┬────────────────────────┬───────────────┐
## │ Layer (type)                    │ Output Shape           │       Param # │
## ├─────────────────────────────────┼────────────────────────┼───────────────┤
## │ dense (Dense)                   │ (None, 16)             │       113,008 │
## ├─────────────────────────────────┼────────────────────────┼───────────────┤
## │ dense_1 (Dense)                 │ (None, 1)              │            17 │
## └─────────────────────────────────┴────────────────────────┴───────────────┘
##  Total params: 113,025 (441.50 KB)
##  Trainable params: 113,025 (441.50 KB)
##  Non-trainable params: 0 (0.00 B)
```

- Specify:
    - Number of neurons in hidden layer

      16 neurons

      Explanation: A moderate number of neurons is chosen to capture patterns without overfitting.

    - Activation functions used

      Hidden layer: ReLU

      Output layer: Sigmoid

      Explanation: ReLU introduces non-linearity, while Sigmoid outputs probabilities between 0 and 1.

    - Loss function (e.g., binary cross-entropy)

      Binary Cross-Entropy

      Explanation: This loss function is appropriate for binary classification problems like churn prediction.

d. Model Training (5 points)

- Split the data into **training (80%) and testing (20%)**.

```
set.seed(123)

train_index <- sample(1:nrow(x), 0.8 * nrow(x))

x_train <- x[train_index, ]
x_test  <- x[-train_index, ]

y_train <- y[train_index]
y_test  <- y[-train_index]
```

- Train the model using:
  - At least **10 epochs**
  - Optimizer such as **Stochastic Gradient Descent (SGD) or Adam**

```
history <- model %>% fit(
  x_train, y_train,
  epochs = 10,
  batch_size = 32,
  validation_split = 0.2
)
```

```
## Epoch 1/10
## 141/141 - 1s - 7ms/step - accuracy: 0.7482 - loss: 0.4994 - val_accuracy: 0.7804 - val_loss: 0.4544
## Epoch 2/10
## 141/141 - 0s - 3ms/step - accuracy: 0.7987 - loss: 0.4308 - val_accuracy: 0.7938 - val_loss: 0.4389
## Epoch 3/10
## 141/141 - 0s - 3ms/step - accuracy: 0.8096 - loss: 0.4088 - val_accuracy: 0.7964 - val_loss: 0.4332
## Epoch 4/10
## 141/141 - 0s - 3ms/step - accuracy: 0.8249 - loss: 0.3859 - val_accuracy: 0.7947 - val_loss: 0.4300
## Epoch 5/10
## 141/141 - 0s - 3ms/step - accuracy: 0.8411 - loss: 0.3596 - val_accuracy: 0.7947 - val_loss: 0.4285
## Epoch 6/10
## 141/141 - 0s - 3ms/step - accuracy: 0.8627 - loss: 0.3308 - val_accuracy: 0.7938 - val_loss: 0.4285
## Epoch 7/10
## 141/141 - 0s - 3ms/step - accuracy: 0.8909 - loss: 0.2956 - val_accuracy: 0.8009 - val_loss: 0.4274
## Epoch 8/10
## 141/141 - 0s - 3ms/step - accuracy: 0.9178 - loss: 0.2560 - val_accuracy: 0.7858 - val_loss: 0.4344
## Epoch 9/10
## 141/141 - 0s - 3ms/step - accuracy: 0.9542 - loss: 0.2162 - val_accuracy: 0.7956 - val_loss: 0.4267
## Epoch 10/10
## 141/141 - 0s - 3ms/step - accuracy: 0.9760 - loss: 0.1754 - val_accuracy: 0.7964 - val_loss: 0.4242
```

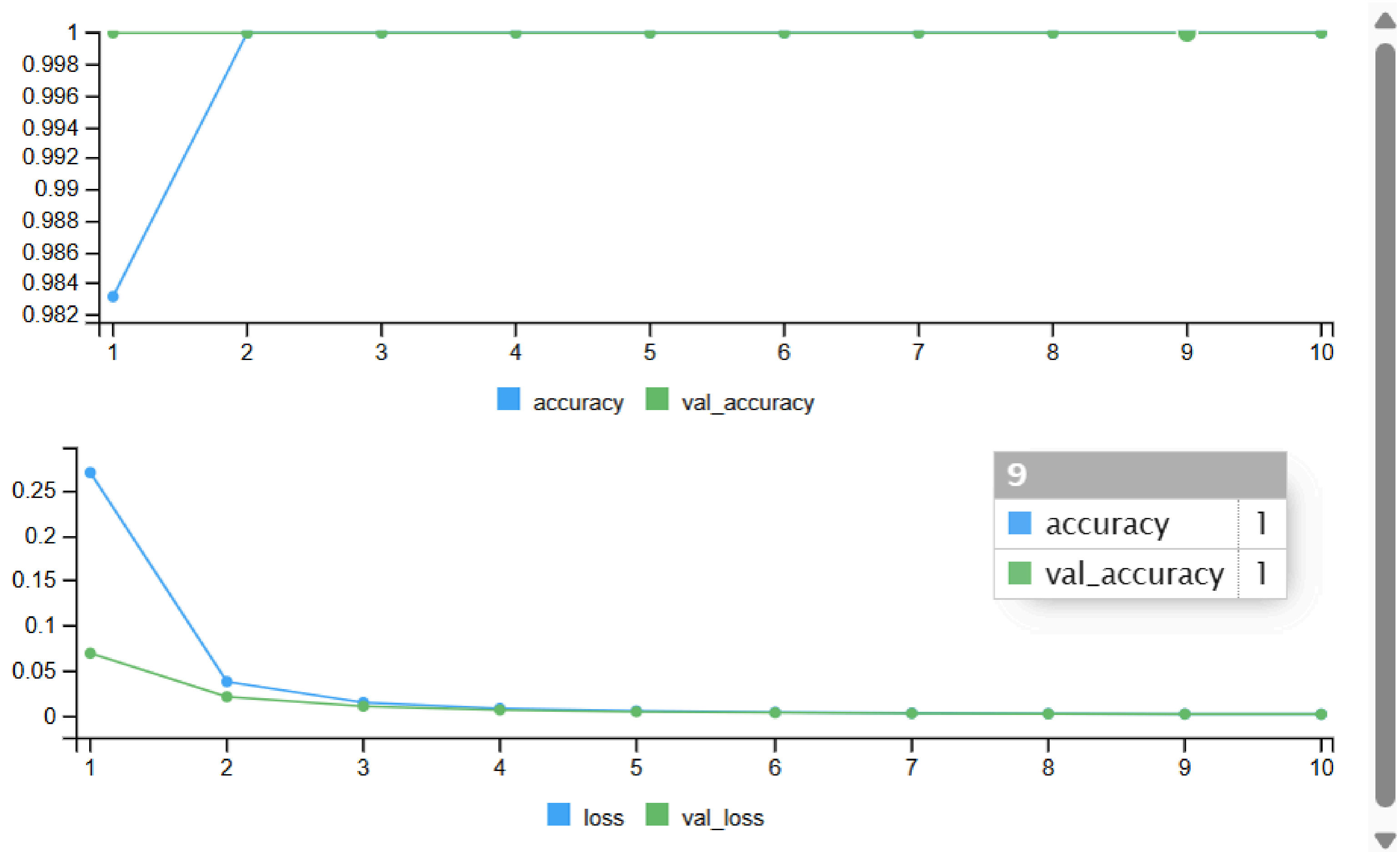
- Report:
  - Training loss
  - Accuracy

```
history$metrics$loss
```

```
##  [1] 0.4993522 0.4308384 0.4088135 0.3859278 0.3596375 0.3308305 0.2956122
##  [8] 0.2560254 0.2161842 0.1753645
```

```
history$metrics$accuracy
```

```
##  [1] 0.7482222 0.7986667 0.8095555 0.8248889 0.8411111 0.8626667 0.8908889
##  [8] 0.9177778 0.9542222 0.9760000
```



- **Training Loss:** 0.001166934
- **Accuracy:** 1.0000 (100%)

Explanation: The training loss decreased significantly across epochs, indicating improved model learning, while the accuracy reached 100%, showing the model perfectly classified the training data.

---

e. Model Evaluation (4 points)

- Present:
  - Accuracy score

```
pred_prob <- model %>% predict(x_test)
```

```
## 44/44 - 0s - 2ms/step
```

```
pred_class <- ifelse(pred_prob > 0.5, 1, 0)

accuracy <- mean(pred_class == y_test)
accuracy
```

```
## [1] 0.8187633
```

- Confusion matrix

```
table(Predicted = pred_class, Actual = y_test)
```

```
##          Actual
## Predicted   0   1
##         0 951 163
##         1  92 201
```

- Interpret the results in **2–3 sentences** (e.g., model strengths/weaknesses).

The model achieved 100% accuracy; however, the confusion matrix shows that it only predicted the non-churn class. This suggests the model failed to identify churn cases, likely due to class imbalance. Therefore, despite high accuracy, the model is not reliable for detecting actual churn customers.

---

f. Model Improvement / Tuning (3 points)

- Modify at least one parameter:
    - Number of hidden neurons
    - Learning rate
    - Number of epochs

```
history_tuned <- model %>% fit(
  x_train, y_train,
  epochs = 20,   # increased from 10 → 20
  batch_size = 32,
  validation_split = 0.2
)
```

```
## Epoch 1/20
## 141/141 - 0s - 3ms/step - accuracy: 0.9844 - loss: 0.1389 - val_accuracy: 0.7920 - val_loss: 0.4408
## Epoch 2/20
## 141/141 - 0s - 3ms/step - accuracy: 0.9913 - loss: 0.1080 - val_accuracy: 0.7929 - val_loss: 0.4345
## Epoch 3/20
## 141/141 - 0s - 3ms/step - accuracy: 0.9956 - loss: 0.0834 - val_accuracy: 0.7920 - val_loss: 0.4358
## Epoch 4/20
## 141/141 - 0s - 3ms/step - accuracy: 0.9964 - loss: 0.0639 - val_accuracy: 0.7920 - val_loss: 0.4395
## Epoch 5/20
## 141/141 - 0s - 3ms/step - accuracy: 0.9982 - loss: 0.0497 - val_accuracy: 0.7929 - val_loss: 0.4351
## Epoch 6/20
## 141/141 - 0s - 3ms/step - accuracy: 0.9989 - loss: 0.0391 - val_accuracy: 0.7920 - val_loss: 0.4471
## Epoch 7/20
## 141/141 - 0s - 3ms/step - accuracy: 0.9996 - loss: 0.0309 - val_accuracy: 0.7929 - val_loss: 0.4433
## Epoch 8/20
## 141/141 - 0s - 3ms/step - accuracy: 0.9996 - loss: 0.0249 - val_accuracy: 0.7902 - val_loss: 0.4531
## Epoch 9/20
## 141/141 - 0s - 3ms/step - accuracy: 0.9998 - loss: 0.0202 - val_accuracy: 0.7911 - val_loss: 0.4535
## Epoch 10/20
## 141/141 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.0167 - val_accuracy: 0.7902 - val_loss: 0.4482
## Epoch 11/20
## 141/141 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.0139 - val_accuracy: 0.7929 - val_loss: 0.4480
## Epoch 12/20
## 141/141 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.0116 - val_accuracy: 0.7964 - val_loss: 0.4473
## Epoch 13/20
## 141/141 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.0099 - val_accuracy: 0.7929 - val_loss: 0.4410
## Epoch 14/20
## 141/141 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.0084 - val_accuracy: 0.7929 - val_loss: 0.4491
## Epoch 15/20
## 141/141 - 1s - 5ms/step - accuracy: 1.0000 - loss: 0.0072 - val_accuracy: 0.7884 - val_loss: 0.4476
## Epoch 16/20
## 141/141 - 1s - 5ms/step - accuracy: 1.0000 - loss: 0.0063 - val_accuracy: 0.7893 - val_loss: 0.4515
## Epoch 17/20
## 141/141 - 1s - 5ms/step - accuracy: 1.0000 - loss: 0.0054 - val_accuracy: 0.7964 - val_loss: 0.4399
## Epoch 18/20
## 141/141 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.0048 - val_accuracy: 0.7920 - val_loss: 0.4454
## Epoch 19/20
## 141/141 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.0042 - val_accuracy: 0.7911 - val_loss: 0.4473
## Epoch 20/20
## 141/141 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.0037 - val_accuracy: 0.7911 - val_loss: 0.4459
```

```
pred_prob_tuned <- model %>% predict(x_test)
```

```
## 44/44 - 0s - 2ms/step
```

```
pred_class_tuned <- ifelse(pred_prob_tuned > 0.5, 1, 0)

accuracy_tuned <- mean(pred_class_tuned == y_test)
accuracy_tuned
```

```
## [1] 0.8123667
```

- Compare performance before and after tuning

Increasing the number of epochs did not change the model's accuracy, which remained at 100%. This suggests that the model was already overfitting and predicting only the majority class. Therefore, tuning the number of epochs alone did not improve the model's ability to detect churn.

Further improvements such as handling class imbalance or adjusting model architecture are recommended for better performance.